

Home Assignment - Chunking Playground - Web URL Chunking

Assignment 2: Web URL Chunking

1. Fixed-Size Character Chunking

The screenshot shows the 'Chunking Playground' interface with the 'Web URL' tab selected. The 'Web URL Ingestion' section on the left shows a URL 'https://developers.cloudflare.com/work' being fetched. The 'Chunking Strategy' section shows 'Fixed-Size Character Chunking' selected with a chunk size of 500 and an overlap of 50. The 'Document Preview' section shows the fetched content from Cloudflare Workers. The 'Analysis' section on the right shows statistics: 6 total chunks, 481.8 average chunk size, 723 estimated tokens, and 0.753 time elapsed. The 'Teaching Mode' section provides information about understanding chunking strategies.

Chunking Playground
VISUALIZING RAG DOCUMENT SEGMENTATION

Web URL Ingestion
Enter a website URL to fetch and extract its primary readable content, stripping boilerplate layout.
<https://developers.cloudflare.com/work> [Fetch](#)

Chunking Strategy
Fixed-Size Character Chunking
Chunk Size (chars): 500
Overlap (chars): 50
[Apply Chunking](#)
[Ingest into MongoDB](#)

Document Preview
Skip to contentDocumentation IndexFetch the complete documentation index at: <https://developers.cloudflare.com/workers/lms.txt>Use this file to discover all available pages before exploring further.
Cloudflare Workers
Last updated Apr 23, 2026(Copy as Markdown|View as Markdown|Agent setup)
A serverless platform for building, deploying, and scaling apps across Cloudflare's global network / with a single command — no infrastructure to manage, no complex configuration
With Cloudflare Workers, you can expect to:
- Deliver fast performance with high reliability anywhere in the world
- Build full-stack apps with your framework of choice, including React, Vue, Svelte, Next, Astro, React Router, and more
- Use your preferred language, including JavaScript, TypeScript, Python, Rust, and more
- Gain deep visibility and insight with built-in observability
363 words 2641 characters 44 36 sentences 56 paragraphs 561 tokens

Analysis
TOTAL CHUNKS: 6
AVG. CHUNK SIZE: 481.8 chars
EST. TOKENS: 723
TIME ELAPSED: 0.753 ms

Teaching Mode
Understand Chunking Strategies
In RAG (Retrieval-Augmented Generation), chunking is the process of breaking input text into smaller segments. Choosing the right size and technique ensures the LLM receives highly relevant context.

The screenshot shows the 'Fixed-Size Chunking Educational Insights' section. It includes a 'Boundary Violations & Risk' section with metrics for sentence boundaries broken (5 times), word boundaries broken (4 times), and context loss rate (High). A 'World Semantic Chunking perform better?' section explains why semantic chunking is better. The 'Chunk Timeline & Overlap Visualizer' shows a timeline of 6 chunks with their start and end positions and overlaps.

FIXED-SIZE CHUNKING EDUCATIONAL INSIGHTS

Boundary Violations & Risk
Sentence Boundaries Broken: 5 times
Word Boundaries Broken: 4 times
Context Loss Rate: High

World Semantic Chunking perform better?
Yes! Here is why:
Semantic Similarity Chunking splits text at natural sentence boundaries based on topic coherence. It avoids breaking sentences in half, preventing context loss and broken words, resulting in superior RAG retrieval precision and recall.

Chunk Timeline & Overlap Visualizer
Document: 2891 chars
Chunk #1: 500 chars
Chunk #2: 500 chars
Chunk #3: 500 chars
Chunk #4: 500 chars
Chunk #5: 500 chars
Chunk #6: 500 chars

The screenshot shows the detailed view of two chunks. Chunk #1 starts at 0 and ends at 500, with an overlap of 0. It contains a warning for 'Sentence Boundary Broken'. Chunk #2 starts at 450 and ends at 950, with an overlap of 50. It contains warnings for 'Word Boundary Broken' and 'Sentence Boundary Broken'. The 'Overlap Region' for Chunk #2 shows the overlapping text: 'no complex configuration' and 'With Cloudflare Workers,'.

Chunk #1
Characters: 0-500
Start: 0 End: 500 Overlap: 0 chars
Sentence Boundary Broken
Skip to contentDocumentation IndexFetch the complete documentation index at: <https://developers.cloudflare.com/workers/lms.txt>Use this file to discover all available pages before exploring further. # Cloudflare Workers Last updated Apr 23, 2026(Copy as Markdown|View as Markdown|Agent setup) A serverless platform for building, deploying, and scaling apps across Cloudflare's global network / with a single command — no infrastructure to manage, no complex configuration With Cloudflare Workers,
A 500 ch 62 w 2 sent 5 para 125 tok

Chunk #2
Characters: 450-950
Start: 450 End: 950 Overlap: 50 chars
Word Boundary Broken **Sentence Boundary Broken**
Overlap Region
OVERLAP START
no complex configuration
With Cloudflare Workers,
OVERLAP END
no complex configuration With Cloudflare Workers, you can expect to: - Deliver fast performance with high reliability anywhere in the world - Build full-stack apps with your framework of choice, including React, Vue, Svelte, Next, Astro, React Router, and more - Use your preferred language, including JavaScript, TypeScript, Python, Rust, and more - Gain deep visibility and insight with built-in observability - Get started for free and grow with flexible pricing, affordable at any scale
A 500 ch 76 w 1 sent 8 para 125 tok

Chunk #3

500 chars

Characters: 988-1488

Start: 900 End: 1400 Overlap: 50 chars

Word Boundary Broken

Sentence Boundary Broken

Overlap Region

OVERLAP START

with flexible pricing, affordable at any scale

Go

OVERLAP END

with flexible pricing, affordable at any scale Get started with your first project: Deploy a template Deploy with Wrangler CLI ## Build with Workers #### Front-end applications Deploy static assets to Cloudflare's CDN & cache for fast rendering #### Back-end applications Build APIs and connect to data stores with Smart Placement to optimize latency #### Serverless AI inference Run LLMs, generate images, and more with Workers AI #### Background jobs Schedule cron jobs, run durable Wor

A 500 ch

70 w

1 sent

13 para

125 tok

Chunk #4

500 chars

Characters: 1358-1858

Start: 1300 End: 1800 Overlap: 50 chars

Word Boundary Broken

Sentence Boundary Broken

Overlap Region

OVERLAP START

ckground jobs

Schedule cron jobs, run durable Wor

OVERLAP END

ckground jobs Schedule cron jobs, run durable Workflows, and integrate with Queues #### Observability & monitoring Monitor performance, debug issues, and analyze traffic with real-time logs and analytics ## Integrate with Workers Connect to external services like databases, APIs, and storage via Bindings, enabling functionality with just a few lines of code: Storage Durable Objects Scalable stateful storage for real-time coordination. D1 Serverless SQL database built for fast, global q

A 500 ch

70 w

2 sent

13 para

125 tok

Chunk #5

500 chars

Characters: 1898-2398

Start: 1800 End: 2300 Overlap: 50 chars

Word Boundary Broken

Sentence Boundary Broken

Overlap Region

OVERLAP START

Serverless SQL database built for fast, global q

OVERLAP END

Serverless SQL database built for fast, global queries. KV Low-latency key-value storage for fast, edge-cached reads. Queues Guaranteed delivery with no charges for egress bandwidth. Hyperdrive Connect to your external database with accelerated queries, cached at the edge. Compute Workers AI Machine learning models powered by serverless GPUs. Workflows Durable, long-running operations with automatic retries. Vectorize Vector database for AI-powered semantic search. R2 Zero-egres

A 500 ch

64 w

6 sent

16 para

125 tok

Chunk #6

391 chars

Characters: 2258-2641

Start: 2250 End: 2641 Overlap: 50 chars

Overlap Region

OVERLAP START

se for AI-powered semantic search.

R2

Zero-egres

OVERLAP END

se for AI-powered semantic search. R2 Zero-egress object storage for cost-efficient data access. Browser Run Programmatic serverless browser instances. Media Cache / CDN Global caching for high-performance, low-latency delivery. Images Streamlined image infrastructure from a single API. Want to connect with the Workers community? Join our Discord / Was this helpful? Yes&#Odoocs

A 391 ch

52 w

8 sent

13 para

98 tok

2. Recursive Character Chunking

Chunking Playground

VISUALIZING RAG DOCUMENT SEGMENTATION

Download Study Guide

Reset Workspace

Backend Online

Upload File

Web URL

Sample Web

Web URL Ingestion

Enter a website URL, to fetch and extract its primary readable content, stripping boilerplate layout.

https://developers.cloudflare.com/work

Fetch

Chunking Strategy

Recursive Character Chunking

Chunk Size (chars) 500

Overlap (chars) 50

Document Preview

Run Analysis

BLOG

Skip to content

Documentation Index

Fetch the complete documentation index at: <https://developers.cloudflare.com/workers/lms.txt>

Use this file to discover all available pages before exploring further.

Cloudflare Workers

Last updated Apr 23, 2026

Copy as Markdown

View as Markdown

Agent setup

A serverless platform for building, deploying, and scaling apps across Cloudflare's global network - with a single command - no infrastructure to manage, no complex configuration

With Cloudflare Workers, you can expect to:

- Deliver fast performance with high reliability anywhere in the world

- Build full-stack apps with your framework of choice, including React, Vue, Svelte, Next, Astro, React Router, and more

- Use your preferred language, including JavaScript, TypeScript, Python, Rust, and more

- Gain deep visibility and insight with built-in observability

H 363 words

A 2641 characters

S 16 sentences

P 56 paragraphs

T 541 tokens

Analysis

Segmentation

TOTAL CHUNKS 6

AVG. CHUNK SIZE 441 chars

EST. TOKENS 662

TIME ELAPSED 0.818 ms

Teaching Mode

Recursive Character Chunking

Splits text by a hierarchical list of characters (typically paragraphs "¶", single newlines "\n", spaces " ", and empty strings ""). It aims to keep paragraphs and sentences intact as much as possible.

Overview

Applications

Practices

Q&A Prep

Advantages

Maintains natural reading structures like paragraphs and sentences.

Highly customizable splitting hierarchies.

Industry standard baseline for text document preprocessing.

Disadvantages

Chunk size can fluctuate based on text formatting.

Still limited to character/token limits without evaluating meaning.

Struggles with heavily structured tabular data.

Chunks View

Comparison Dashboard

RAG Simulator

Proceed to Ingest into HangoDB

Tree View

Grid View

Recursive Character Chunking Educational Insights

Hierarchy Preservation Metrics

Paragraph Preservation %: 100%

Line Preservation %: 100%

Word Breaks Count: 0 breaks

Character Splits (Forced): 0 splits

Separator Usage Analytics

Paragraph (¶) 5

Line (\n) 0

Word (" ") 0

Char ("") 0

Fixed vs Recursive Split Comparison

Metric	Fixed Chunking	Recursive Chunking
Paragraph Preservation	83.3%	100%
Line Preservation	92%	100%
Word Boundary Breaks	4	0
Sentence Boundary Breaks	5	0

Dynamic Hierarchy Preservation

Why is Recursive better than Fixed?

Fixed-size chunking splits text strictly by character length, which frequently splits words (e.g., comput-er) and cuts sentences in half. Recursive chunking uses a hierarchical fallback system: it attempts splits on paragraphs first, then lines, then word spaces. This ensures word boundaries are never broken and sentences remain complete inside chunks.

Why is Recursive STILL not Semantic?

While recursive chunking preserves structural text segments (like paragraphs and lines), it does not analyze the semantic meaning or content transitions of sentences. If a single topic spans across paragraphs, recursive splitting might partition them into separate chunks purely based on size limits, whereas Semantic Chunking evaluates topic embedding similarities to group related paragraphs coherently.

Recursive Process Walking Simulator

Show Recursive Process

Chunk #1

474 chars

EXCELLENT SPLIT

Split Separator Used: \n

Split Reason: Chunk size reached at paragraph boundary. (Level: Paragraph)

Here

Line

Word

Char

Skip to content

Documentation Index

Fetch the complete documentation index at: <https://developers.cloudflare.com/workers/lms.txt>

Use this file to discover all available pages before exploring further.

Cloudflare Workers

Last updated Apr 23, 2026

Copy as Markdown

View as Markdown

Agent setup

A serverless platform for building, deploying, and scaling apps across Cloudflare's global network - with a single command - no infrastructure to manage, no complex configuration

A 474 ch

S 39 w

P 2 sent

T 4 para

T 118 tok

Chunk #2

470 chars

EXCELLENT SPLIT

Split Separator Used: \n

Split Reason: Chunk size reached at paragraph boundary. (Level: Paragraph)

Here

Line

Word

Char

With Cloudflare Workers, you can expect to:

- Deliver fast performance with high reliability anywhere in the world

- Build full-stack apps with your framework of choice, including React, Vue, Svelte, Next, Astro, React Router, and more

- Use your preferred language, including JavaScript, TypeScript, Python, Rust, and more

- Gain deep visibility and insight with built-in observability

- Get started for free and grow with flexible pricing, affordable at any scale

A 470 ch

S 72 w

P 1 sent

T 6 para

T 118 tok

Chunk #3

485 chars

EXCELLENT SPLIT

Split Separator Used: \n

Split Reason: Chunk size reached at paragraph boundary. (Level: Paragraph)

Here

Line

Word

Char

Get started with your first project: Deploy a template

Deploy with Wrangler CLI

Build with Workers

Front-end applications

Deploy static assets to Cloudflare's CDN & cache for fast rendering

Back-end applications

Build APIs and connect to data stores with Smart Placement to optimize latency

Serverless AI inference

Run LLMs, generate images, and more with Workers AI

Background jobs

Schedule cron jobs, run durable Workflows, and integrate with Queues

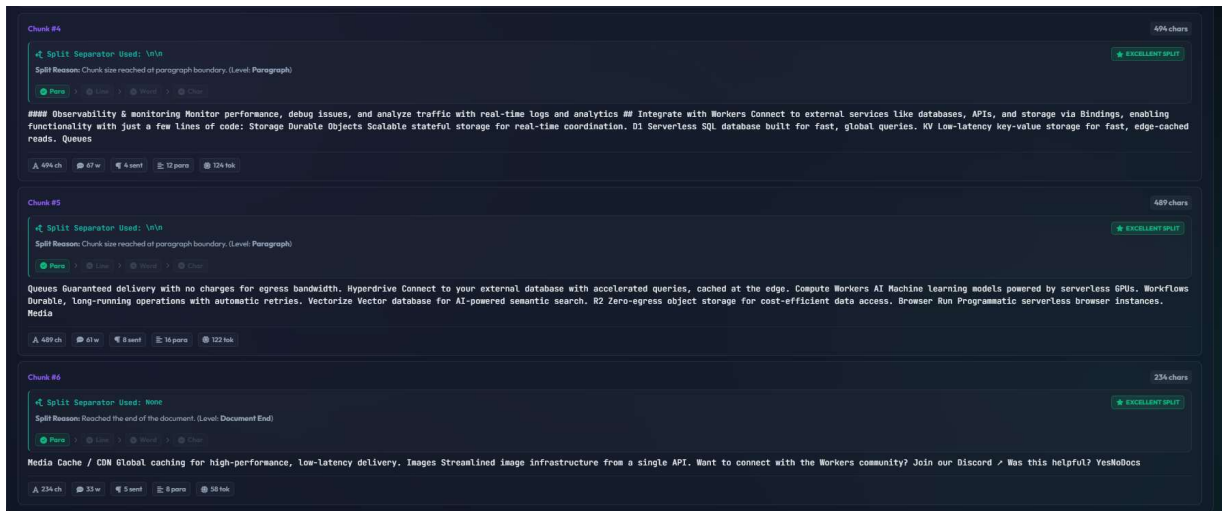
A 485 ch

S 73 w

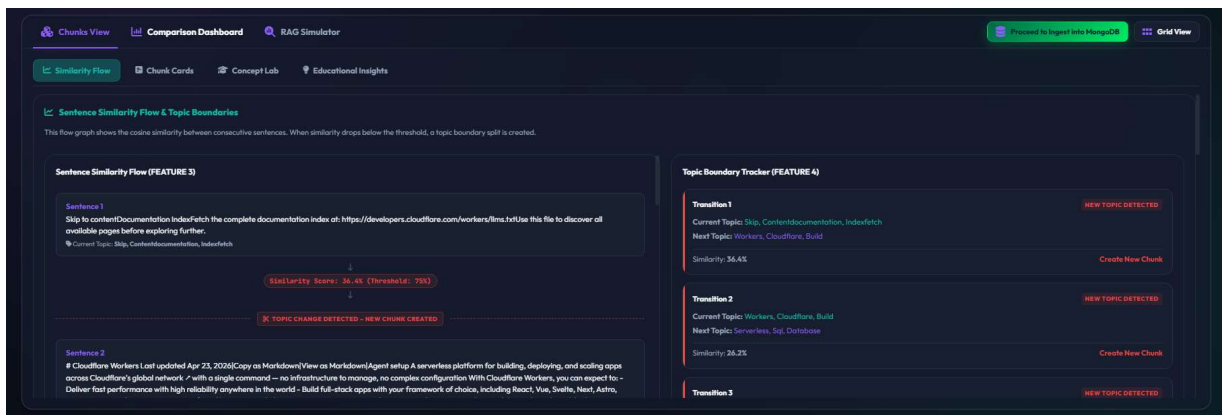
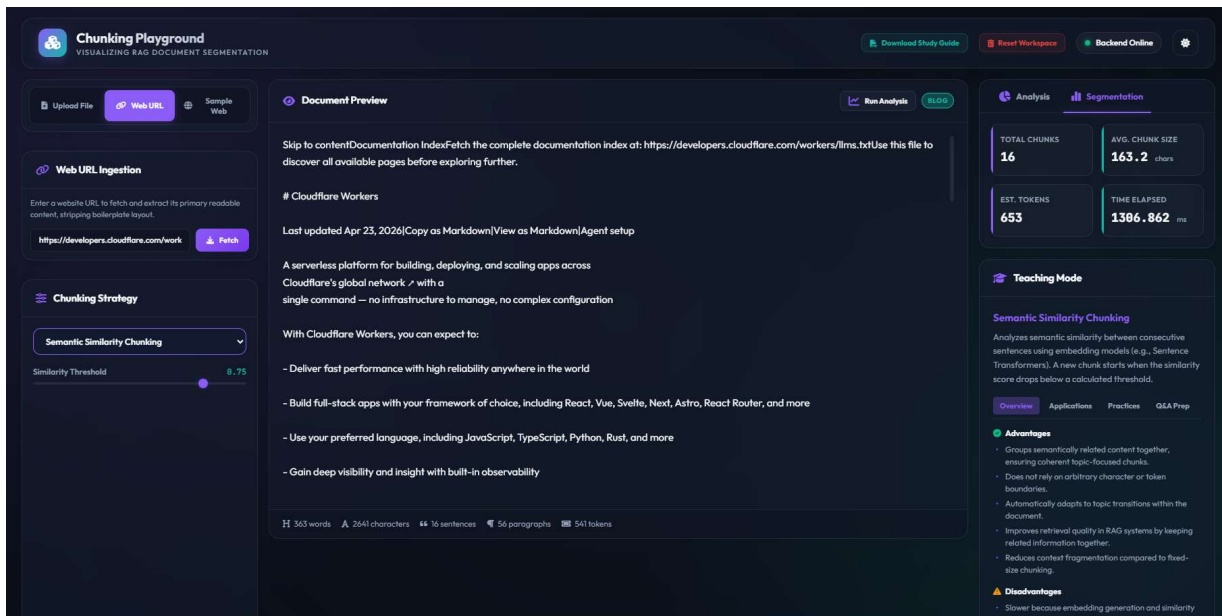
P 1 sent

T 12 para

T 121 tok



3. Semantic Similarity Chunking



⌕ TOPIC CHANGE DETECTED - NEW CHUNK CREATED

Sentence 2

Cloudflare Workers Last updated Apr 23, 2026Copy as MarkdownView as MarkdownAgent setup A serverless platform for building, deploying, and scaling apps across Cloudflare's global network - with a single command - no infrastructure to manage, no complex configuration With Cloudflare Workers, you can expect to - Deliver fast performance with high reliability anywhere in the world - Build full-stack apps with your framework of choice, including React, Vue, Svelte, Next, Astro, React Router, and more - Use your preferred language, including JavaScript, TypeScript, Python, Rust, and more - Gain deep visibility and insight with built-in observability - Get started for free and grow with flexible pricing, affordable at any scale Get started with your first project: Deploy a template Deploy with Wrangler CLI ## Build with Workers #### Front-end applications Deploy static assets to Cloudflare's CDN & cache for fast rendering #### Back-end applications Build APIs and connect to data stores with Smart Placement to optimize latency #### Serverless AI Inference Run LLMs, generate images, and more with Workers AI #### Background jobs Schedule cron jobs, run durable Workflows, and integrate with Queues #### Observability & monitoring Monitor performance, debug issues, and analyze traffic with real-time logs and analytics ## Integrate with Workers Connect to external services like databases, APIs, and storage via Bindings, enabling functionality with just a few lines of code Storage Durable Objects Scalable stateful storage for real-time coordination.

📄 Current Topic: Workers, Cloudflare, Build

Transition 2

Current Topic: Workers, Cloudflare, Build
Next Topic: Serverless, Sql, Database

Similarity: 26.2%

Create New Chunk

Transition 3

Current Topic: Serverless, Sql, Database
Next Topic: Low, Latency, Key

Similarity: 38.1%

Create New Chunk

Transition 4

Current Topic: Low, Latency, Key
Next Topic: Queues, Guaranteed, Delivery

Similarity: 38.6%

Create New Chunk

Transition 5

Current Topic: Queues, Guaranteed, Delivery
Next Topic: Hyperdrive, Connect, External

Similarity: 28.4%

Create New Chunk

Transition 6

Current Topic: Hyperdrive, Connect, External
Next Topic: Compute, Workers, Machine

Similarity: 18.7%

Create New Chunk

Transition 7

Current Topic: Compute, Workers, Machine
Next Topic: Workflows, Durable, Long

Similarity: 22.8%

Create New Chunk

4. Metadata-Enhanced

Upload File

Web URL

Sample Web

Web URL Ingestion

Enter a website URL to fetch and extract its primary readable content, stripping boilerplate layout.

https://developers.cloudflare.com/work

Fetch

Chunking Strategy

Metadata-Enhanced

Chunk Size (chars) 500

Overlap (chars) 50

Metadata Settings

☒ Auto-Discover (Heuristic)

Apply Chunking

Ingest into MongoDB

Document Preview

Run Analysis

SLUG

Skip to contentDocumentation IndexFetch the complete documentation index at: https://developers.cloudflare.com/workers/lms.txtUse this file to discover all available pages before exploring further.

Cloudflare Workers

Last updated Apr 23, 2026Copy as MarkdownView as MarkdownAgent setup

A serverless platform for building, deploying, and scaling apps across Cloudflare's global network / with a single command — no infrastructure to manage, no complex configuration

With Cloudflare Workers, you can expect to:

- Deliver fast performance with high reliability anywhere in the world

- Build full-stack apps with your framework of choice, including React, Vue, Svelte, Next, Astro, React Router, and more

- Use your preferred language, including JavaScript, TypeScript, Python, Rust, and more

- Gain deep visibility and insight with built-in observability

363 words 2641 characters 16 sentences 56 paragraphs 541 tokens

Analysis

Segmentation

TOTAL CHUNKS 18

AVG. CHUNK SIZE 155.1 chars

EST. TOKENS 698

TIME ELAPSED 4.921 ms

Teaching Mode

Metadata-Enhanced Chunking

Embeds vital context headers directly into each text segment (e.g. department, version, parent document name) before indexing.

OverviewApplicationsPracticesQ&A Prep

Advantages

- Allows precise post-retrieval filtering.
- Maintains global context inside micro-segments.

Disadvantages

- Increases token count of each chunk.
- Requires structured metadata extraction pipeline.

Why Companies Use It

- Enables precise filtering by departments, authors, or document versions.
- Allows vector databases to boost matches containing important keywords.
- Preserves document context that is otherwise lost during row text extraction.

Chunks View

Comparison Dashboard

RAG Simulator

Proceed to Ingest into MongoDB

Grid View

Chunk Cards

Timeline Flow

Strategy Comparison

Search metadata values...

All Departments

Clear Filters

Chunk #1

Topic: Cloudflare / Workers / Markdown

TRK: Standard Operating Procedure

Doc: Compliance

Lang: English

Depth: Quality Assurance

Reg: Global

Secs: Internal

Ver: 1.0

Word: 363

Est: 660

Targ: Standard Users

500 chars

Context Prepend: [Dept: Quality Assurance | Ver: 1.0 | Section: Introduction]

Skip to contentDocumentation IndexFetch the complete documentation index at: https://developers.cloudflare.com/workers/lms.txtUse this file to discover all available pages before exploring further. # Cloudflare Workers Last updated Apr 23, 2026Copy as MarkdownView as MarkdownAgent setup A serverless platform for building, deploying, and scaling apps across Cloudflare's global network / with a single command — no infrastructure to manage, no complex configuration With Cloudflare Workers,

Chunk #2

Topic: React / Including / More

TRK: Standard Operating Procedure

Doc: Compliance

Lang: English

Depth: Quality Assurance

Reg: Global

Secs: Internal

Ver: 1.0

Word: 363

Est: 660

Targ: Standard Users

300 chars

Context Prepend: [Dept: Quality Assurance | Ver: 1.0 | Section: Introduction]

no complex configuration With Cloudflare Workers, you can expect to: - Deliver fast performance with high reliability anywhere in the world - Build full-stack apps with your framework of choice, including React, Vue, Svelte, Next, Astro, React Router, and more - Use your preferred language, including JavaScript, TypeScript, Python, Rust, and more - Gain deep visibility and insight with built-in observability - Get started for free and grow with flexible pricing, affordable at any scale Ge

Chunk #3

Topic: Pricing / Flexible / Project

TRK: Standard Operating Procedure

Doc: Compliance

Lang: English

Depth: Quality Assurance

Reg: Global

Secs: Internal

Ver: 1.0

Word: 363

Est: 660

Targ: Standard Users

86 chars

Context Prepend: [Dept: Quality Assurance | Ver: 1.0 | Section: Introduction]

with flexible pricing, affordable at any scale Get started with your first project:

Chunk #4

Topic: Template / Deploy

TRK: Standard Operating Procedure

Doc: Compliance

Lang: English

Depth: Quality Assurance

Reg: Global

Secs: Internal

Ver: 1.0

Word: 363

Est: 660

Targ: Standard Users

19 chars

Context Prepend: [Dept: Quality Assurance | Ver: 1.0 | Section: Deploy a template]

Deploy a template

Chunk #5	Topic: Jobs / Build / Workers	TIR: Standard Operating Procedure	Doc: Compliance	Lang: English	Depth: Quality Assurance	Regi: Global	Secu: Internal	Ver: 1.0	Word: 363	Esti: 660	Targ: Standard Users	500 chars
Context Prepend: [Dept: Quality Assurance Ver: 1.0 Section: Deploy with Wrangler CLI] Deploy with Wrangler CLI ## Build with Workers #### Front-end applications Deploy static assets to Cloudflare's CDN & cache for fast rendering #### Back-end applications Build APIs and connect to data stores with Smart Placement to optimize latency #### Serverless AI Inference Run LLMs, generate images, and more with Workers AI #### Background jobs Schedule cron jobs, run durable Workflows, and integrate with Queues #### Observability & monitoring Monitor performance, debug issues, an												
Chunk #6	Topic: Monitor / External / Code	TIR: Standard Operating Procedure	Doc: Compliance	Lang: English	Depth: Quality Assurance	Regi: Global	Secu: Internal	Ver: 1.0	Word: 363	Esti: 660	Targ: Standard Users	262 chars
Context Prepend: [Dept: Quality Assurance Ver: 1.0 Section: Deploy with Wrangler CLI] monitoring Monitor performance, debug issues, and analyze traffic with real-time logs and analytics ## Integrate with Workers Connect to external services like databases, APIs, and storage via Bindings, enabling functionality with just a few Lines of code:												
Chunk #7	Topic: Storage	TIR: Standard Operating Procedure	Doc: Compliance	Lang: English	Depth: Quality Assurance	Regi: Global	Secu: Internal	Ver: 1.0	Word: 363	Esti: 660	Targ: Standard Users	9 chars
Context Prepend: [Dept: Quality Assurance Ver: 1.0 Section: Storage] Storage												
Chunk #8	Topic: Storage / Fast / Object	TIR: Standard Operating Procedure	Doc: Compliance	Lang: English	Depth: Quality Assurance	Regi: Global	Secu: Internal	Ver: 1.0	Word: 363	Esti: 660	Targ: Standard Users	797 chars
Context Prepend: [Dept: Quality Assurance Ver: 1.0 Section: Durable Objects] Durable Objects Scalable stateful storage for real-time coordination. D1 Serverless SQL database built for fast, global queries. KV Low-latency key-value storage for fast, edge-cached reads.												
Chunk #9	Topic: Bandwidth / Queues / Delivery	TIR: Standard Operating Procedure	Doc: Compliance	Lang: English	Depth: Quality Assurance	Regi: Global	Secu: Internal	Ver: 1.0	Word: 363	Esti: 660	Targ: Standard Users	47 chars
Context Prepend: [Dept: Quality Assurance Ver: 1.0 Section: Queues] Queues Guaranteed delivery with no charges for egress bandwidth.												
Chunk #10	Topic: External / Hyperdrive / Edge	TIR: Standard Operating Procedure	Doc: Compliance	Lang: English	Depth: Quality Assurance	Regi: Global	Secu: Internal	Ver: 1.0	Word: 363	Esti: 660	Targ: Standard Users	93 chars
Context Prepend: [Dept: Quality Assurance Ver: 1.0 Section: Hyperdrive] Hyperdrive Connect to your external database with accelerated queries, cached at the edge.												
Chunk #11	Topic: Compute	TIR: Standard Operating Procedure	Doc: Compliance	Lang: English	Depth: Quality Assurance	Regi: Global	Secu: Internal	Ver: 1.0	Word: 363	Esti: 660	Targ: Standard Users	9 chars
Context Prepend: [Dept: Quality Assurance Ver: 1.0 Section: Compute] Compute												
Chunk #12	Topic: Machine / Gpus / Learning	TIR: Standard Operating Procedure	Doc: Compliance	Lang: English	Depth: Quality Assurance	Regi: Global	Secu: Internal	Ver: 1.0	Word: 363	Esti: 660	Targ: Standard Users	45 chars
Context Prepend: [Dept: Quality Assurance Ver: 1.0 Section: Workers AI] Workers AI Machine learning models powered by serverless GPUs.												
Chunk #13	Topic: Retries / Workflows / Operations	TIR: Standard Operating Procedure	Doc: Compliance	Lang: English	Depth: Quality Assurance	Regi: Global	Secu: Internal	Ver: 1.0	Word: 363	Esti: 660	Targ: Standard Users	69 chars
Context Prepend: [Dept: Quality Assurance Ver: 1.0 Section: Workflows] Workflows Durable, long-running operations with automatic retries.												
Chunk #14	Topic: Vectorize / Semantic / Database	TIR: Standard Operating Procedure	Doc: Compliance	Lang: English	Depth: Quality Assurance	Regi: Global	Secu: Internal	Ver: 1.0	Word: 363	Esti: 660	Targ: Standard Users	124 chars
Context Prepend: [Dept: Quality Assurance Ver: 1.0 Section: Vectorize] Vectorize Vector database for AI-powered semantic search. R2 Zero-egress object storage for cost-efficient data access.												
Chunk #15	Topic: Browser / Programatic / Instances	TIR: Standard Operating Procedure	Doc: Compliance	Lang: English	Depth: Quality Assurance	Regi: Global	Secu: Internal	Ver: 1.0	Word: 363	Esti: 660	Targ: Standard Users	57 chars
Context Prepend: [Dept: Quality Assurance Ver: 1.0 Section: Browser Run] Browser Run Programatic serverless browser instances.												
Chunk #16	Topic: Media / Cache / Performance	TIR: Standard Operating Procedure	Doc: Compliance	Lang: English	Depth: Quality Assurance	Regi: Global	Secu: Internal	Ver: 1.0	Word: 363	Esti: 660	Targ: Standard Users	80 chars
Context Prepend: [Dept: Quality Assurance Ver: 1.0 Section: Media] Media Cache / CDN Global caching for high-performance, low-latency delivery.												
Chunk #17	Topic: Streamlined / Wood / Helpful	TIR: Standard Operating Procedure	Doc: Compliance	Lang: English	Depth: Quality Assurance	Regi: Global	Secu: Internal	Ver: 1.0	Word: 363	Esti: 660	Targ: Standard Users	145 chars
Context Prepend: [Dept: Quality Assurance Ver: 1.0 Section: Images] Images Streamlined image infrastructure from a single API. Want to connect with the Workers community? Join our Discord / Was this helpful?												
Chunk #18	Topic: YesNoDocs	TIR: Standard Operating Procedure	Doc: Compliance	Lang: English	Depth: Quality Assurance	Regi: Global	Secu: Internal	Ver: 1.0	Word: 363	Esti: 660	Targ: Standard Users	9 chars
Context Prepend: [Dept: Quality Assurance Ver: 1.0 Section: YesNoDocs] YesNoDocs												

5. LLM-Based Intelligent

The screenshot displays the 'Chunking Playground' interface, a tool for visualizing RAG document segmentation. The interface is divided into several sections:

- Top Bar:** Includes a logo, the title 'Chunking Playground', and a subtitle 'VISUALIZING RAG DOCUMENT SEGMENTATION'. It also features links for 'Download Study Guide', 'Reset Workspace', and 'Backend Online'.
- Left Panel:**
 - Web URL Ingestion:** A section for entering a website URL to fetch and extract its primary readable content, with a 'Fetch' button.
 - Chunking Strategy:** A dropdown menu set to 'LLM-Based Intelligent'. Below it, a 'Target Chunk Size (chars)' is set to 588. There is a checkbox for 'Simulate LLM (No Key Required)' which is checked.
 - Apply Chunking:** A button at the bottom of the left panel.
- Document Preview:** A central area showing the document content. It includes a 'Run Analysis' button and a 'BLOB' button. The preview shows text from a document about Cloudflare Workers, including a URL and a description of the platform.
- Analysis / Segmentation:** A section on the right showing analysis results:
 - TOTAL CHUNKS:** 56
 - AVG. CHUNK SIZE:** 45.2 chars
 - EST. TOKENS:** 633
 - TIME ELAPSED:** 2.495 ms
- Teaching Mode:** A section on the right titled 'LLM-Based Intelligent Chunking' which explains the process and lists advantages and disadvantages.

The bottom section of the image shows a 'Chunks View' with a list of 9 chunks, each with a topic and a simulated LLM response. The chunks are:

- Chunk #1: Topic Document / Skip / Skip. Simulated LLM: 198 chars.
- Chunk #2: Topic Document / Workers / Cloudflare. Simulated LLM: 20 chars.
- Chunk #3: Topic Document / Markdown / Copy. Simulated LLM: 71 chars.
- Chunk #4: Topic Document / Manage / Configuration. Simulated LLM: 179 chars.
- Chunk #5: Topic Document / Workers / Cloudflare. Simulated LLM: 43 chars.
- Chunk #6: Topic Document / Anywhere / Performance. Simulated LLM: 70 chars.
- Chunk #7: Topic Document / React / Svelte. Simulated LLM: 120 chars.
- Chunk #8: Topic Document / More / Rust. Simulated LLM: 87 chars.
- Chunk #9: Topic Document / Built / Gain. Simulated LLM: 42 chars.

Chunk #10	Logtail Section: Get started for free and grow w/	Topic: Document / Pricing / Flexible	Simulated LLM	78 chars
- Get started for free and grow with flexible pricing, affordable at any scale				
Chunk #11	Logtail Section: Get started with your first project	Topic: Document / Project / Started	Simulated LLM	36 chars
Get started with your first project:				
Chunk #12	Logtail Section: Deploy a template	Topic: Document / Template / Deploy	Simulated LLM	17 chars
Deploy a template				
Chunk #13	Logtail Section: Deploy with Wrangler CLI	Topic: Document / Wrangler / Deploy	Simulated LLM	24 chars
Deploy with Wrangler CLI				
Chunk #14	Logtail Section: ## Build with Workers	Topic: Document / Workers / Build	Simulated LLM	21 chars
## Build with Workers				
Chunk #15	Logtail Section: #### Front-end applications	Topic: Document / Applications / Front	Simulated LLM	27 chars
#### Front-end applications				
Chunk #16	Logtail Section: Deploy static assets to Cloudflare	Topic: Document / Cache / Static	Simulated LLM	67 chars
Deploy static assets to Cloudflare's CDN & cache for fast rendering				
Chunk #17	Logtail Section: #### Back-end applications	Topic: Document / Applications / Back	Simulated LLM	26 chars
#### Back-end applications				
Chunk #18	Logtail Section: Build APIs and connect to data s/b	Topic: Document / Placement / Stores	Simulated LLM	78 chars
Build APIs and connect to data stores with Smart Placement to optimize latency				
Chunk #19	Logtail Section: #### Serverless AI Inference	Topic: Document / Inference / Serverless	Simulated LLM	28 chars
#### Serverless AI Inference				

Mongo DB Ingestion Process:

**MongoDB Ingestion**

Partitioned Document & Chunk Storage





Ingestion & Embedding Complete

Each chunk has been stored as an individual MongoDB document with a 1,024-dimensional Mistral embedding.

TARGET COLLECTION

web_url_chunks

STORAGE ARCHITECTURE

Individual Chunk Documents

EMBEDDING MODEL

Mistral (1,024 Dimensions)

VECTOR SEARCH STATUS

Ready (vector_index)

DOCUMENT NAME

developers.cloudflare.com

DOCUMENTS INGESTED

56 Documents

FIRST INSERTED ID

6ab80311013d2853743469e1

 **Verify Vector Search**

1,024-d Cosine Similarity

 **Search**

Close

ORGANIZATION
vigneshwarani's Org

PROJECT
Project 0

Compare tiers

?

My Queries

Data Modeling

CLUSTERS (1)

Search clusters

Cluster0

ragTest_cases

Resume_RAG

admin

chunking_playground

document_chunks

resume_chunks

web_url_chunks

local

web_url_chunks

Cluster0 > chunking_playground > web_url_chunks

Documents 168

Aggregations

Schema

Indexes 1

Validation

Type a query: { field: 'value' } or [Generate query](#)

ADD DATA

BULK

EXPORT CODE

25

1 - 26 of 168

_id: ObjectId("6ab7ed57c829d93784987a49")

document_id: "7ff737e1c-c917-48b9-8875-aa440843378b"

document_name: "developers.cloudflare.com"

source_type: "web_url"

source_url: "https://developers.cloudflare.com/workers/"

strategy: "llm"

strategy_params: Object (2)

chunk_id: 1

chunk_index: 1

total_chunks: 56

text: "Skip to contentDocumentation IndexFetch the complete documentation index at: https://developers.cloudflare.com/workers/llm.txtUse this file to discover all available pages before exploring..."

char_count: 198

start_char: null

end_char: null

overlap_size: 0

metadata: Object (18)

embedding: Array (1024)

embedding_model: "mistral-embed"

embedding_dimensions: 1024

ingested_at: "2026-09-26T16:05:43.191765+08:00"

_id: ObjectId("6ab7ed57c829d93784987a49")

document_id: "7ff737e1c-c917-48b9-8875-aa440843378b"

document_name: "developers.cloudflare.com"

source_type: "web_url"

source_url: "https://developers.cloudflare.com/workers/"

strategy: "llm"

Alert System Status: All Good

©2026 MongoDB, Inc. [Status](#) [Terms](#) [Privacy](#) [Atlas Blog](#) [Contact Sales](#)

60